

Introducere în programare - curs și laborator - suport electronic

(C) 2023-2024 Bogdan Pătruț

Meniu

[Lectia 1](#) | [Lectia 2](#) | [Lectia 3](#) | [Lectia 4](#) | [Lectia 5](#) | [Lectia 6](#) | [Lectia 7](#)
[Material pentru laboratoare](#) | [Material pentru laboratorul de grafica](#)

Lectia 6 - Elemente de grafică 2D

În acest curs vom face o introducere foarte simplă în domeniul graficii 2D. De fapt, vom rezolva niște probleme de geometrie analitică, folosind o reprezentare carteziană xOy , care apoi va putea fi transpusă în reprezentări grafice pe ecranul calculatorului, la [laborator](#).

Problemele prezentate în acest curs vor fi rezolvate plecând de la următoarele premise:

1. vom lucra în cadranul 1 al sistemului de axe (abscise pozitive, ordinate pozitive);
2. originea sistemului de axe (punctul de coordonate $(0, 0)$) se află în colțul din stânga sus al ecranului;
3. sistemul de axe este oglindit pe verticală față de reprezentarea clasică de la matematică; cu alte cuvinte, axa Ox este orientată de la stânga la dreapta (de la colțul stânga-sus la colțul dreapta sus), iar axa Oy este orientată de sus în jos (de la colțul stânga-sus la colțul stânga-jos);
4. spațiul de lucru este discret, coordonatele fiind numere naturale, dar toate calculele le vom face cu numere reale, care se pot converti (prin rotunjire sau trunchiere) la numere naturale;
5. dispunem de o bibliotecă grafică ce conține o primitivă grafică pentru trasarea unui segment de dreaptă între două puncte $P(x_1, y_1)$ și $P(x_2, y_2)$: `line(x1, y1, x2, y2)`;
6. vom folosi funcțiile trigonometrice (`sin`, `cos` etc.) cu argumentul dat în radiani.

Video

- Exemplu simplu de utilizare a graficii: [click aici](#)
- Desenarea unui poligon regulat: [click aici](#)
- Desenarea unei stele: [click aici](#)
- Figuri recursive: [click aici](#)
- Graficul unei funcții (21:33): [click aici](#)

Scopul nostru este de a realiza diferite exerciții de grafică, în care accentul să-l punem pe partea matematică și de algoritmică și nu pe tehnologie sau pe elemente de programare orientată-obiect (care vor fi predate la alte cursuri). De aceea, vom utiliza o interfață grafică simplă, la care noi putem adăuga propriile funcții mai complexe, create pe baza unor funcții simple predefinite. Așadar, vom folosi cea mai simplă bibliotecă grafică, numită **WinBGIm**, care extinde funcțiile din **Graphics.h**.

Scopul nostru nu este de a folosi biblioteci grafice sofisticate, pentru a face efecte de grafică tridimensională sau jocuri cu un design grafic foarte frumos.

Pentru asemenea lucruri, puteți folosi alte biblioteci grafice precum **Dislin**, **SDL**, **SFML**, **OpenGL**, **GLUT** etc. Folosirea acestor biblioteci grafice pentru dezvoltarea de jocuri nu este interzisă, dar fiind mult mai complicate decât **WinBGIm**, nu putem oferi ajutor de niciun fel și rămâne la latitudinea voastră să vă

documentații singuri.

Studentii cu experiență în instalarea și utilizarea bibliotecilor grafice **SDL**, **SFML** sau **Dislin** (nu și **OpenGL** sau **GLUT**) sunt încurajați să contacteze pe dl. profesor Bogdan Pătruț pentru include în website-ul laboratorului ghiduri simplificate de instalare a lor pentru CodeBlocks 20 și sistemul de operare Windows.

Atenție! Biblioteca grafică va fi necesară și pentru unele din proiectele din partea a doua a semestrului. **Nota nu crește** dacă proiectul este vizual mult mai frumos, dacă partea de interfață este făcută cu ajutorul unor **componente/obiecte prefabricate**, de genul butoane, casete de selecție, liste simple sau combinate, ferestre de dialog sau alte **elemente GUI predefinite**.

Dimpotrivă, **studentii sunt încurajați să-și definească ei propriile elemente de interfață grafică**, folosind propriile structuri de date și bazându-se pe primitivele grafice elementare puse la dispoziție de o bibliotecă grafică cât mai simplă, cum este **WinBGIm**.

6.1. O bibliotecă grafică elementară

Biblioteca grafică WinBGIm se poate descărca de aici: [WINBGIM.ZIP](#). Atenție: pe web puteți găsi și alte variante ale acestei arhive, dar nefuncționale.

Instalarea bibliotecii se face după indicațiile din laboratorul 6:

[CLICK AICI](#)

Descrierea funcțiilor grafice

- Lista funcțiilor grafice, cu exemple la fiecare:
<https://home.cs.colorado.edu/~main/cs1300/doc/bgi/index.html>
- Alte lucruri: <http://www.cs.colorado.edu/~main/cs1300/doc/bgi/bgi.html>.

ATENȚIE!! WinBGIm este o bibliotecă grafică pentru Windows. Pentru macOS, Linux, încercați o bibliotecă grafică similară, cum ar fi SDL_bgi, pe care o găsiți aici: <http://libxbgi.sourceforge.net/>

Un exemplu de program de test pentru grafica este aici:

```
#include <iostream>
#include <graphics.h>
#include <winbgim.h>

int main()
{
    // int gd = DETECT, gm;
    // initgraph(&gd, &gm, "");
    initwindow(800,600); int x = 400, y = 300, r;
    for (r = 25; r <= 125; r += 20)
        circle(x, y, r);
    getch(); closegraph(); return 0;
}
```

Dacă acesta funcționează, puteți continua să folosiți orice funcție grafică și de lucru cu mouse-ul din biblioteca WinBGIm.

6.2. Prezentări video

Următoarele lecții video se referă în primul rând la elementele de matematică necesare reprezentărilor grafice. Urmăriți-le cu răbdare și cu atenție.

- Exemplu simplu de utilizare a graficii (10:16): [click aici](#)
- Desenarea unui poligon regulat (12:11): [click aici](#)
- Desenarea unei stele regulate (5:22): [click aici](#)
- Figuri recursive (11:51): [click aici](#)
- Graficul unei funcții (21:33): [click aici](#)

Click [aici pentru a merge direct la materialul de laborator.](#)

15 grafica simpla



16 poligon regulat



17 stea regulata



18 diamant



24 grafic functie



Pentru a dezvolta programe grafice, iată alte trei exemple mai jos.

Pe lângă elementele de mai jos, vedeți și miniproiectele din laborator. Click [aici pentru a merge direct la materialul de laborator](#).

6.3. Program demonstrativ pentru utilizarea graficii și a mouse-ului

```
#include <iostream>
#include <winbgim.h>
#include <stdlib.h>
using namespace std;

int stanga,sus,width,height,latura, numar;
bool gata;

void punerePiesa()
{
    int linia,coloana,x,y;
    if(ismouseclick(WM_LBUTTONDOWN))
    {
        clearmouseclick(WM_LBUTTONDOWN);
        x=mousex();
        y=mousey();
        if (!(x>=stanga && x<=stanga+width && y>=sus&&y<=sus+height))
            gata=true;
        else
        {
            linia=(y-sus)/latura+1;
            coloana=(x-stanga)/latura+1;
            circle(stanga+latura*(coloana-1)+latura/2,sus+latura*(linia-1)+latura/2,10);
        }
    }
}

void desen()
{
    int i,j;
    numar=5;
    width=400;
    height=400;
    latura=width/numar;
    sus=(getmaxy()-width)/2;
    stanga=(getmaxx()-height)/2;
    for(i=1; i<=numar; i++)
        for(j=1; j<=numar; j++)
            rectangle(stanga+latura*(i-1),sus+latura*(j-1),stanga+latura*i,sus+latura*j);
}

int main()
{
    initwindow(800,600);
    desen();
    gata=false;
    do
    {
        punerePiesa();
    }
    while (!gata);
    getch();
    closegraph();
    return 0;
}
```

6.4. Un exemplu de joc cu o matrice pentru reprezentarea situației jocului

Prin click de mouse într-una din căsuțele libere ale tablei de joc, apare întâi un cerc roz. La al doilea click apare un pătrat galben. La al treilea click, spațiul se eliberează. „Scorul” este de fapt numărul de pătrate verzi de pe tabla de joc. Când se face click în afara tablei, procesul încetează. Așadar, la final va trebui să faceți click în afara tablei de joc, iar apoi să apăsați o tastă oarecare, pentru a ieși din program. Programul este pur didactic, nu poate fi considerat un joc real. El exemplifică utilizarea unor funcții grafice, desenarea unei table de joc pătrate, folosirea mouse-ului, precum și asocierea unei matrice tablei de joc.

```
#include <iostream>
#include <winbgim.h>
#include <stdlib.h>

using namespace std;

#define MAX 20
#define FUNDAL CYAN

int stanga,sus,width,height,latura, numar;
bool gata;

int TablaDeJoc[MAX][MAX];

int scor;

void afiseazaScor()
{
    char sirScor[10];
    setcolor(FUNDAL); itoa(scor,sirScor,10);
    outtextxy(200,50,sirScor);
    int i,j; scor=0;
    for (i=1;i<=numar;i++)
        for (j=1;j<=numar;j++)
            if (TablaDeJoc[i][j]==2) scor++;
    setcolor(BLUE); itoa(scor,sirScor,10);
    outtextxy(200,50,sirScor);
}

void punerePiesa()
{
    int linia,coloana,x,y;
    int x1, y1, x2, y2;
    int xmijloc, ymijloc;
    if(ismouseclick(WM_LBUTTONDOWN))
    {
        clearmouseclick(WM_LBUTTONDOWN);
        x=mousex(); y=mousey();
        if (!(x>=stanga && x<=stanga+width && y>=sus&&y<=sus+height))
            gata=true;
        else
        {
            linia=(y-sus)/latura+1; coloana=(x-stanga)/latura+1;
            if (TablaDeJoc[linia][coloana]==0)
            {
                TablaDeJoc[linia][coloana]=1;
                // stergere
                setcolor(FUNDAL); setfillstyle(SOLID_FILL,FUNDAL);
                x1=stanga+latura*(coloana-1); y1=sus+latura*(linia-1);
                x2=x1+latura; y2=y1+latura;
                xmijloc=(x1+x2)/2; ymijloc=(y1+y2)/2;
                bar(xmijloc-20,ymijloc-20,xmijloc+20,ymijloc+20);
            }
        }
    }
}
```

```

        // afisez cerc rosu aprins
        setcolor(LIGHTRED); setfillstyle(SOLID_FILL,LIGHTRED);
        fillellipse(xmijloc,ymijloc,18,18);
    }
    else
    if (TablaDeJoc[linia][coloana]==1)
    {
        TablaDeJoc[linia][coloana]=2;
        setcolor(FUNDAL); setfillstyle(SOLID_FILL,FUNDAL);
        x1=stanga+latura*(coloana-1); y1=sus+latura*(linia-1);
        x2=x1+latura; y2=y1+latura;
        xmijloc=(x1+x2)/2; ymijloc=(y1+y2)/2;
        bar(xmijloc-20,ymijloc-20,xmijloc+20,ymijloc+20);
        // afisez un patrat galben
        setcolor(LIGHTRED); setfillstyle(SOLID_FILL,YELLOW);
        bar(xmijloc-20,ymijloc-20,xmijloc+20,ymijloc+20);
    }
    else // 2-> 0
    {
        TablaDeJoc[linia][coloana]=0;
        setcolor(FUNDAL); setfillstyle(SOLID_FILL,FUNDAL);
        x1=stanga+latura*(coloana-1); y1=sus+latura*(linia-1);
        x2=x1+latura; y2=y1+latura;
        xmijloc=(x1+x2)/2; ymijloc=(y1+y2)/2;
        bar(xmijloc-20,ymijloc-20,xmijloc+20,ymijloc+20);
    }
    afiseazaScor();
}
}
}

void desen()
{
    int i,j; numar=5;
    width=400; height=400; latura=width/numar;
    sus=(getmaxy()-width)/2; stanga=(getmaxx()-height)/2;
    setbkcolor(FUNDAL); clearviewport(); setcolor(BLUE);
    for (i=1; i<=numar; i++)
        for (j=1; j<=numar; j++)
            rectangle(stanga+latura*(i-1),sus+latura*(j-1),stanga+latura*i,sus+latura*j);
}

int main()
{
    initwindow(800,600); scor=0; desen(); afiseazaScor();
    gata=false;
    do
    {
        punerePiesa();
    }
    while (!gata);
    getch(); closegraph();
    return 0;
}

```

6.5. Reprezentarea grafică a arborilor

O problema interesanta este problema reprezentării grafice a unui arbore și si noi vom prezenta unele idei de rezolvare. Reprezentarea va apela la metoda "divide et impera" și va face apel la mai multe funcții care vor determina înălțimea arborelui, numărul de frunze, „lățimea” arborelui și altele. De asemenea, studenții vor vedea modalitatea în care trebuie să calculeze coordonatele nodurilor arborelui, în funcție de zonele de desenare ale fiecărui subarbore.

Video

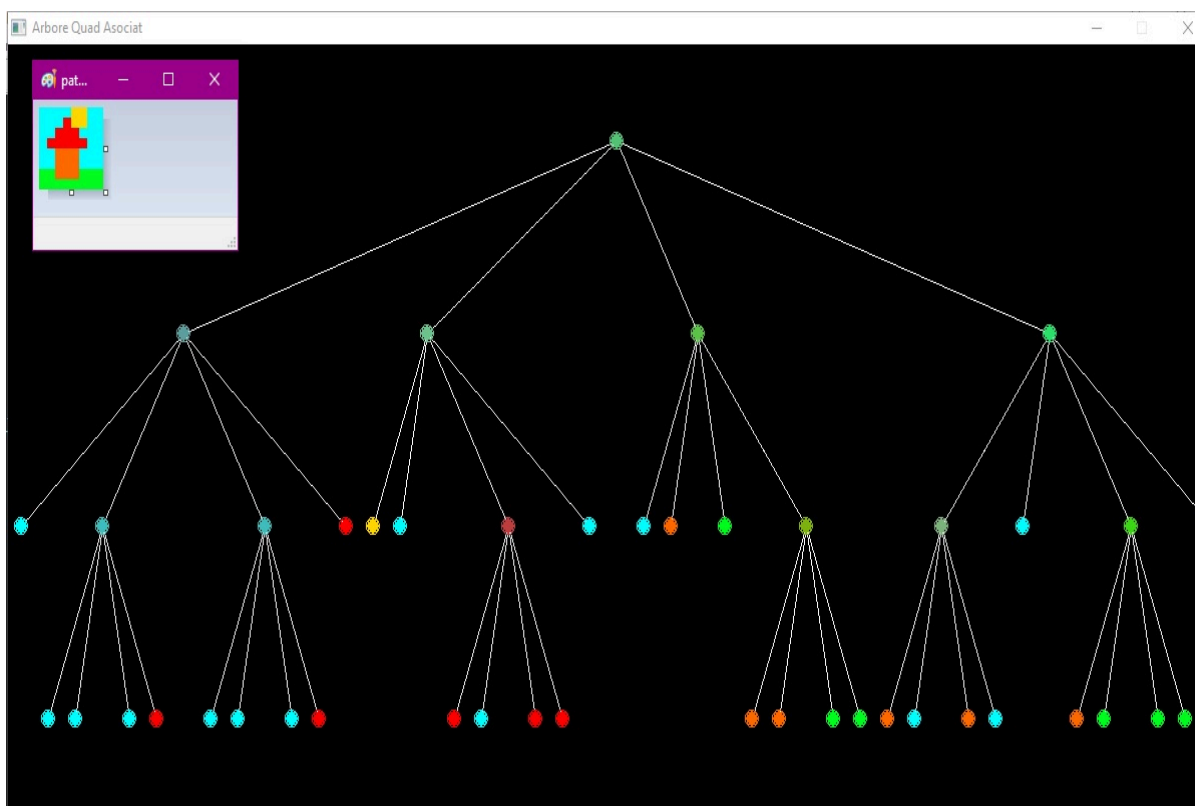
- Ideea de bază pentru reprezentarea grafică a unui arbore binar (9:45): [click aici](#)
- Calcul nr. niveluri (înălțime) și lățime arbore (7:24): [click aici](#)
- Desenarea arborelui binar (8:53): [click aici, dar atenție la calculul numărului de coloane necesare subarborelui stang!](#)

6.6. Arbori quad

Un alt subiect interesant este cel al arborilor quad și posibile aplicații ale lor.

Video

- Arbori quad și utilizarea lor pentru reprezentarea imaginilor (20:44): [click aici](#)



Click [aici](#) pentru a merge direct la materialul de laborator.